

Xây dựng RESTful API chuẩn MVC (No Database)

1. **Định nghĩa:** MVC viết tắt của Model-View-Controller, một kiến trúc phổ biến để tổ chức mã nguồn trong dự án một cách hiệu quả. Ví dụ để hiểu đơn giản mô hình này theo nghiệp vụ của một nhà hàng như sau:
 - **Model (Nhà bếp & Kho):** Chịu trách nhiệm về dữ liệu.
 - Trong nhà hàng: Là nơi chứa nguyên liệu (thịt, rau) và công thức nấu ăn.
 - Trong Code: Là nơi kết nối Database, quy định dữ liệu (Schema), lấy/lưu dữ liệu.
 - **View (Món ăn bày lên đĩa):** Chịu trách nhiệm hiển thị.
 - Trong nhà hàng: Là món ăn đã trang trí đẹp mắt mang ra cho khách.
 - Trong Code: Là giao diện HTML (nếu làm web truyền thống) hoặc dữ liệu JSON (nếu làm API) trả về cho người dùng.
 - **Controller (Bồi bàn & Bếp trưởng):** Chịu trách nhiệm xử lý logic.
 - Trong nhà hàng: Nhận order từ khách -> Hô vào bếp lấy nguyên liệu -> Chế biến -> Đưa ra đĩa.
 - Trong Code: Nhận request từ Route -> Gọi Model lấy data -> Xử lý tính toán -> Trả về View/JSON.

2. Cách tổ chức thư mục dự án (Best Practice)

Khi áp dụng MVC, cấu trúc thư mục của sinh viên sẽ chuyển từ "một file server.js hỗn độn" sang cấu trúc ngăn nắp sau:

```
my-project/
├── src/
│   ├── config/db.js          # Cấu hình kết nối Database
│   ├── models/               # (M) Định nghĩa cấu trúc dữ liệu
│   │   ├── User.js           (VD: User.js, Product.js)
│   │   └── Product.js
│   ├── views/                # (V) Chứa file HTML/EJS (Hoặc bỏ
│   │   └── index.ejs          qua nếu chỉ viết API trả JSON)
│   └── controllers/           # (C) Chứa logic xử lý (VD:
│       └── userController.js
```

```

|   └─ routes/           # Định tuyến đường đi (VD:
userRoutes.js)
|   └─ index.js          # File chạy chính (Khởi tạo server)
└─ .env                  # Biến môi trường
└─ package.json

```

Lưuồng đi của dữ liệu: Khách hàng (Request) → Routes (Chỉ đường) → Controller (Xử lý) ↔ Model (Dữ liệu) → Controller → View/JSON (Response) → Khách hàng.

Mục tiêu bài thực hành:

- Hiểu cách chia nhỏ dự án thành các module: Model, View (bỏ qua vì làm API), Controller, Route.
- Thành thạo sử dụng express.Router.
- Quản lý dữ liệu mảng nhưng tổ chức code như dự án thật.

Phần 1: Khởi tạo và Cấu trúc thư mục

Nhiệm vụ: Sinh viên tạo cấu trúc thư mục "chuẩn chỉnh" thay vì viết hết vào 1 file.

1. Tạo dự án mới:

```

mkdir mvc-express-lab
cd mvc-express-lab
npm init -y
npm install express
npm install --save-dev nodemon

```

2. Tạo cây thư mục: (Yêu cầu sinh viên tạo thủ công để nhớ)

```

mvc-express-lab/
└─ package.json
└─ server.js          <-- Cổng chính của ứng dụng
└─ src/
    └─ models/         <-- Nơi chứa dữ liệu (Array)
    └─ controllers/    <-- Nơi xử lý logic (Logic bán hàng)
    └─ routes/         <-- Nơi điều hướng (Menu)

```

Phần 2: Xây dựng Model

Giải thích: Dữ liệu sẽ giả lập được nằm tại Model, bài thực hành sau chúng ta sẽ làm việc với cơ sở dữ liệu thay thế, sử dụng MongoDB.

File: src/models/courseModel.js

```
// Giả lập Database bằng một mảng
const courses = [
  { id: 1, name: 'NodeJS Super Fast', price: 1200000,
    description: 'Học Nodejs từ số 0' },
  { id: 2, name: 'ReactJS Master', price: 1500000,
    description: 'Làm chủ React trong 1 tháng' },
  { id: 3, name: 'Fullstack MERN', price: 2500000,
    description: 'Khóa học từ A-Z' }
];

// Xuất mảng này ra để Controller sử dụng
module.exports = courses;
```

Phần 3: Xây dựng Controller

Giải thích: Controller sẽ nhập data từ Model, xử lý, và trả về kết quả.

File: src/controllers/courseController.js

```
// 1. Import dữ liệu từ Model
const courses = require('../models/courseModel');

// 2. Viết các hàm xử lý logic

// [GET] /api/courses
exports.getAllCourses = (req, res) => {
  res.status(200).json({
    success: true,
```

```
        count: courses.length,
        data: courses
    });
};

// [GET] /api/courses/:id
exports.getCourseById = (req, res) => {
    const id = parseInt(req.params.id);
    const course = courses.find(c => c.id === id);

    if (!course) {
        return res.status(404).json({ success: false, message:
'Không tìm thấy khóa học' });
    }

    res.status(200).json({ success: true, data: course });
};

// [POST] /api/courses
exports.createCourse = (req, res) => {
    const { name, price, description } = req.body;

    // Validate đơn giản
    if (!name || !price) {
        return res.status(400).json({ success: false, message:
'Vui lòng nhập tên và giá' });
    }
}
```

```
// Tạo ID tự tăng
const newId = courses.length > 0 ? courses[courses.length - 1].id + 1 : 1;

const newCourse = {
  id: newId,
  name,
  price,
  description: description || 'Chưa có mô tả'
};

courses.push(newCourse);

res.status(201).json({ success: true, data: newCourse });
};

// [PUT] /api/courses/:id
exports.updateCourse = (req, res) => {
  const id = parseInt(req.params.id);
  const course = courses.find(c => c.id === id);

  if (!course) {
    return res.status(404).json({ success: false, message:
'Không tìm thấy khóa học để sửa' });
  }

  // Cập nhật dữ liệu
  course.name = req.body.name || course.name;
```

```

    course.price = req.body.price || course.price;
    course.description = req.body.description ||
course.description;

    res.status(200).json({ success: true, message: 'Cập nhật
thành công', data: course });
};

// [DELETE] /api/courses/:id
exports.deleteCourse = (req, res) => {
    const id = parseInt(req.params.id);
    const index = courses.findIndex(c => c.id === id);

    if (index === -1) {
        return res.status(404).json({ success: false, message:
'Không tìm thấy khóa học để xóa' });
    }

    courses.splice(index, 1);
    res.status(200).json({ success: true, message: 'Đã xóa thành
công' });
};

```

Phần 4: Xây dựng Router

Giải thích: Router đóng vai trò "người chỉ đường", ghép nối URL với hàm Controller tương ứng.

File: src/routes/courseRoutes.js

```

const express = require('express');
const router = express.Router(); // Khởi tạo Router

```

```
// Import Controller để dùng
const courseController =
require('../controllers/courseController');

// Định nghĩa các đường dẫn
// Gọn gàng hơn nhiều so với viết trong server.js

router.get('/', courseController.getAllCourses);
router.post('/', courseController.createCourse);

router.get('/:id', courseController.getCourseById);
router.put('/:id', courseController.updateCourse);
router.delete('/:id', courseController.deleteCourse);

// Xuất router ra để server.js dùng
module.exports = router;
```

Phần 5: Kết nối tại Server.js

Giải thích: File server.js bây giờ rất sạch sẽ, chỉ làm nhiệm vụ khởi động và nạp các routes.

File: server.js

```
const express = require('express');
const app = express();
const port = 3000;

// Import Routes
const courseRoutes = require('../src/routes/courseRoutes');

// Middleware đọc JSON
```

```
app.use(express.json());

// --- ROUTES ---
// Mount (gắn) route vào đường dẫn gốc /api/courses
app.use('/api/courses', courseRoutes);

// Route mặc định trang chủ
app.get('/', (req, res) => {
    res.send('Chào mừng đến với API quản lý khóa học (MVC)');
});

// Chạy server
app.listen(port, () => {
    console.log(`Server chạy tại: http://localhost:${port}`);
});
```

Phần 6: Kiểm thử và Thử thách

Kiểm thử: Sinh viên dùng **Postman** hoặc **Thunder Client** để test đủ 5 chức năng (CRUD).

Thử thách: *Thêm chức năng: Lọc khóa học theo khoảng giá.*

- **Yêu cầu:** GET /api/courses/filter?min=1000000&max=2000000
- **Gợi ý:**
 1. Vào courseController.js viết thêm hàm filterByPrice.
 2. Dùng req.query.min và req.query.max.
 3. Dùng hàm filter() của Array để lọc.
 4. Vào courseRoutes.js khai báo thêm đường dẫn router.get('/filter', courseController.filterByPrice); (Lưu ý: phải đặt route này nằm *trên* route /:id để tránh xung đột).